

Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 2

Part 1:

Exercise 1: UNIFORM DATA

Part A - Vertex Transformation

Assume a three-dimensional translation by $\langle m, n, p \rangle$, identify each point (x, y, z) with the point $(x, y, z, 1)$, the general form of this transformation in matrix form is:

$$\begin{bmatrix} 1 & 0 & 0 & m \\ 0 & 1 & 0 & n \\ 0 & 0 & 1 & p \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x+m \\ y+n \\ z+p \\ 1 \end{bmatrix}$$

The additional “1” in homogeneous coordinates allows translation to be represented as a matrix multiplication, so that all affine transformations (rotation, scaling, translation) can be combined into a single 4×4 transformation matrix.

Part B – Translation in Animation-1

1.

- We got the first point: $(0.0, 0.2, 1)$. From translation = $[-0.5, 0.0, 0.0]$:

$$T = \begin{bmatrix} 1 & 0 & -0.5 \\ 0 & 1 & 0.0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Apply to the general form:

$$\begin{bmatrix} 1 & 0 & -0.5 \\ 0 & 1 & 0.0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 0.0 \\ 0.2 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 0.2 \\ 1 \end{bmatrix}$$

- The result will be

```

[-0.47, 0.0, 0.0]
[-0.4399999999999995, 0.0, 0.0]
[-0.4099999999999999, 0.0, 0.0]
[-0.3799999999999999, 0.0, 0.0]
[-0.3499999999999987, 0.0, 0.0]
[-0.3199999999999984, 0.0, 0.0]
[-0.289999999999998, 0.0, 0.0]
[-0.259999999999998, 0.0, 0.0]
[-0.229999999999998, 0.0, 0.0]
[-0.199999999999998, 0.0, 0.0]
[-0.169999999999998, 0.0, 0.0]
[-0.139999999999998, 0.0, 0.0]
[-0.1099999999999979, 0.0, 0.0]
[-0.079999999999998, 0.0, 0.0]
[-0.04999999999999795, 0.0, 0.0]
[-0.01999999999999796, 0.0, 0.0]
[0.010000000000000203, 0.0, 0.0]
[0.04000000000000002, 0.0, 0.0]
[0.07000000000000002, 0.0, 0.0]
[0.10000000000000002, 0.0, 0.0]
[0.13000000000000002, 0.0, 0.0]
[0.16000000000000002, 0.0, 0.0]
[0.19000000000000002, 0.0, 0.0]
[0.22000000000000002, 0.0, 0.0]
[0.25000000000000002, 0.0, 0.0]
[0.28000000000000025, 0.0, 0.0]
[0.31000000000000003, 0.0, 0.0]
[0.34000000000000003, 0.0, 0.0]
[0.37000000000000003, 0.0, 0.0]
[0.40000000000000003, 0.0, 0.0]
[0.43000000000000004, 0.0, 0.0]
[0.46000000000000004, 0.0, 0.0]
[0.49000000000000004, 0.0, 0.0]
[0.52000000000000005, 0.0, 0.0]
[0.55000000000000005, 0.0, 0.0]
[0.58000000000000005, 0.0, 0.0]
[0.61000000000000005, 0.0, 0.0]
[0.64000000000000006, 0.0, 0.0]

```

At each frame, the triangle will change the coordinate of x-axis by +0.03. So the triangle will move from left to right on the screen.

2. Why does the program reset the x-translation when $x > 1.2$?

In OpenGL clip space, the value of `gl_Position (x, y, z)` inside `[-1, 1]`

If $x > 1$, it will disappear from the screen.

$\Rightarrow x > 1.2$ means the triangle will run completely out of the screen and appear on the left side of the screen.

Part C – Circular Motion in Animation-2Code:

1. The update rule is:

$$x = r * \cos(\theta), y = r * \sin(\theta)$$

with $r = 0.75$, $\theta = \text{time}$

Derive these equations from the unit circle definition.

$$x(t) = 0.75\cos(\theta),$$

$$y(t) = 0.75\sin(\theta)$$

What is the translation matrix for a point (0,0,1) moved to (x,y) on the circle?

$$T = \begin{bmatrix} 1 & 0 & 0.75\cos(\theta) \\ 0 & 1 & 0.75\sin(\theta) \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.75\cos(\theta) \\ 0.75\sin(\theta) \\ 1 \end{bmatrix}$$

2. Show how the vertex (0.0, 0.2) is transformed at:

$$T = \begin{bmatrix} 1 & 0 & 0.75\cos(\theta) \\ 0 & 1 & 0.75\sin(\theta) \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 0.0 \\ 0.2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.75\cos(\theta) + 0.0 \\ 0.75\sin(\theta) + 0.2 \\ 1 \end{bmatrix}$$

• $t = 0$:

$$T = \begin{bmatrix} 0.75\cos(0) \\ 0.75\sin(0) \\ 1 \end{bmatrix} * \begin{bmatrix} 0 \\ 0.2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.2 \\ 1 \end{bmatrix}$$

• $t = \pi/2$

$$T = \begin{bmatrix} 0.75\cos(\pi/2) \\ 0.75\sin(\pi/2) \\ 1 \end{bmatrix} * \begin{bmatrix} 0 \\ 0.2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.95 \\ 1 \end{bmatrix}$$

- $t = \pi$:

$$T = \begin{bmatrix} 0.75\cos(\pi) \\ 0.75\sin(\pi) \\ 1 \end{bmatrix} * \begin{bmatrix} 0 \\ 0.2 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.75 \\ 0.2 \\ 1 \end{bmatrix}$$

Part D – Implementation:

1. Orbiting & Spinning Triangle

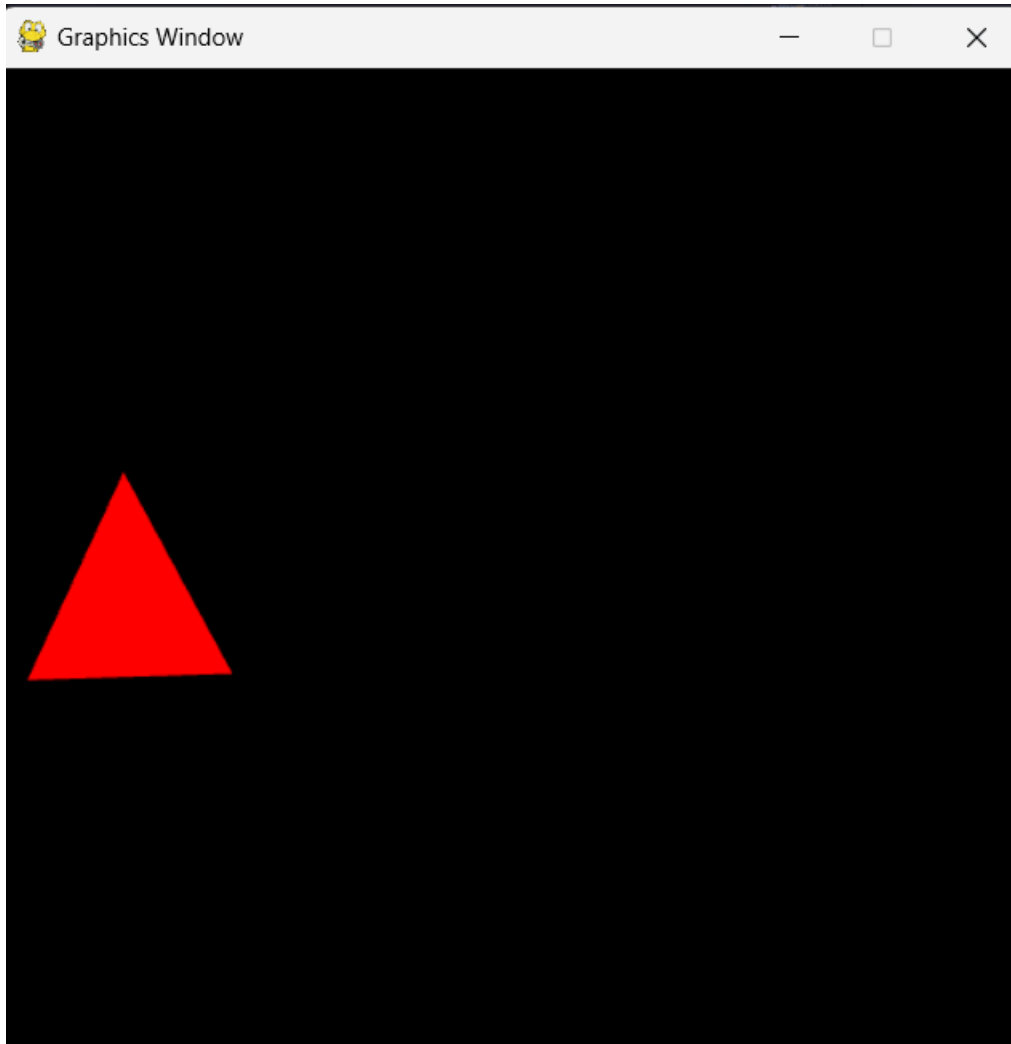
Code:

```

1  #!/usr/bin/python3
2  import math
3
4  import OpenGL.GL as GL
5
6  from py3d.core.base import Base
7  from py3d.core.utils import Utils
8  from py3d.core.attribute import Attribute
9  from py3d.core.uniform import Uniform
10
11
12  class Example(Base):
13      """ Animate a triangle moving in a circular path around the origin """
14      def initialize(self):
15          print("Initializing program...")
16          # Initialize program #
17          vs_code = """
18              in vec3 position;
19              uniform vec3 translation;
20              uniform vec3 center;
21              uniform float angle;
22              void main()
23              {
24                  float c = cos(angle);
25                  float s = sin(angle);
26                  mat2 rotation = mat2(c, -s, s, c);
27
28                  vec2 local = position.xy - center.xy;
29                  vec2 rotated = rotation * local;
30                  vec2 world = rotated + center.xy + translation.xy;
31
32                  gl_Position = vec4(world.x, world.y, position.z + translation.z, 1.0);
33              }
34          """
35          fs_code = """
36              uniform vec3 baseColor;
37              out vec4 fragColor;
38              void main()
39              {
40                  fragColor = vec4(baseColor.r, baseColor.g, baseColor.b, 1.0);
41              }
42          """
43          self.program_ref = Utils.initialize_program(vs_code, fs_code)
44          # Render settings (optional) #
45          # Specify color used when clearing
46          GL.glClearColor(0.0, 0.0, 0.0, 1.0)
47          # Set up vertex array object #
48          vao_ref = GL.glGenVertexArrays(1)
49          GL.glBindVertexArray(vao_ref)
50          # Set up vertex attribute #
51          self.position_data = [[ 0.0, 0.2, 0.0],
52                               [ 0.2, -0.2, 0.0],
53                               [-0.2, -0.2, 0.0]]
54          self.vertex_count = len(self.position_data)
55          position_attribute = Attribute('vec3', self.position_data)
56          position_attribute.associate_variable(self.program_ref, 'position')
57
58          cx = sum(v[0] for v in self.position_data) / self.vertex_count
59          cy = sum(v[1] for v in self.position_data) / self.vertex_count
60          cz = 0.0
61          # Set up uniforms #
62          self.translation = Uniform('vec3', [0.0, 0.0, 0.0])
63          self.translation.locate_variable(self.program_ref, 'translation')
64
65          self.center = Uniform('vec3', [cx, cy, cz])
66          self.center.locate_variable(self.program_ref, 'center')
67
68          self.angle = Uniform('float', 0.0)
69          self.angle.locate_variable(self.program_ref, 'angle')
70
71          self.base_color = Uniform('vec3', [1.0, 0.0, 0.0])
72          self.base_color.locate_variable(self.program_ref, 'baseColor')
73
74          self.orbit_radius = 0.75
75          self.orbit_speed = 1.0 # radians per second
76          self.spin_speed = 2.0
77
78      def update(self):
79          """ Update data """
80          t = self.time
81          self.translation.data[0] = 0.75 * math.cos(self.time)
82          self.translation.data[1] = 0.75 * math.sin(self.time)
83          self.angle.data = self.spin_speed * t
84          # Reset color buffer with specified color
85          GL.glClear(GL.GL_COLOR_BUFFER_BIT)
86          GL.glUseProgram(self.program_ref)
87          self.translation.upload_data()
88          self.center.upload_data()
89          self.angle.upload_data()
90          self.base_color.upload_data()
91          GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)
92
93
94  # Instantiate this class and run the program
95  Example().run()

```

Result:

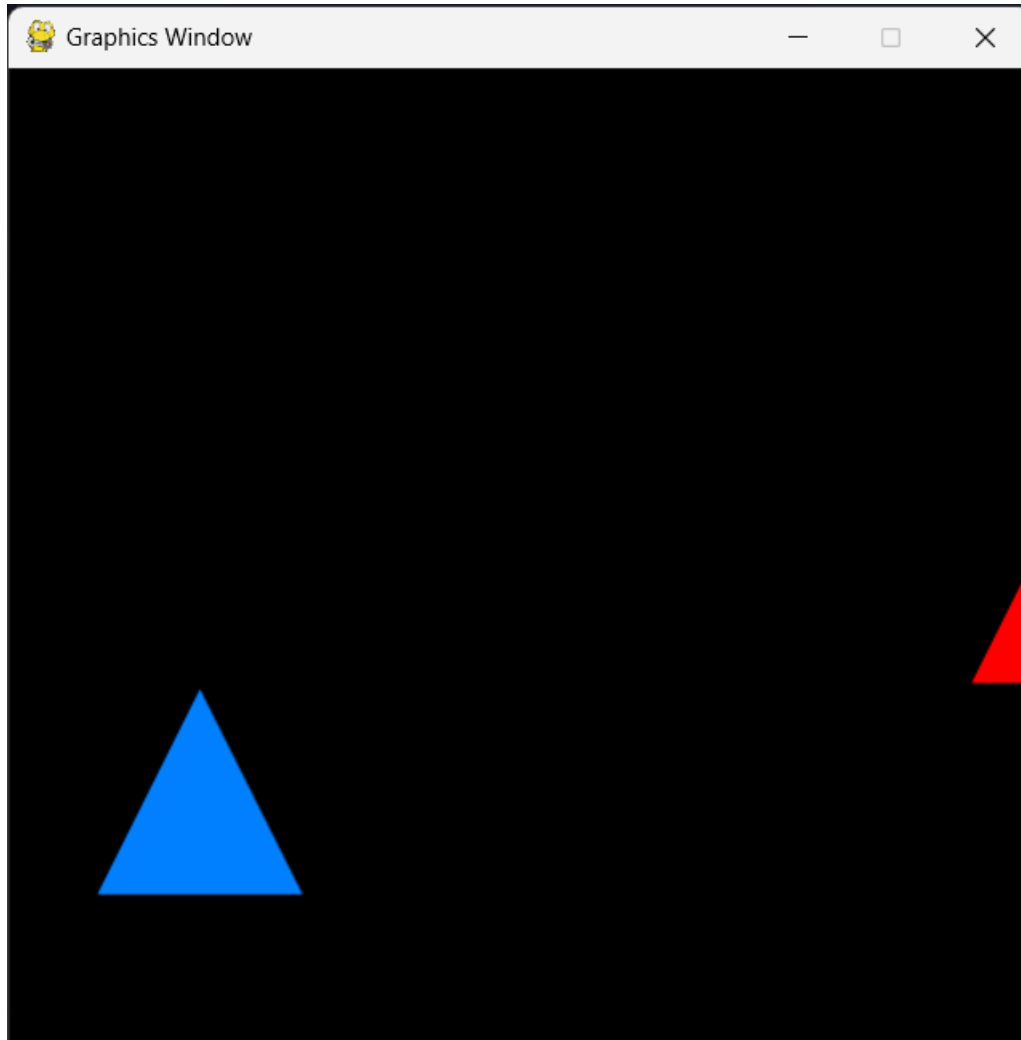


2. Two Triangles (Linear vs Circular Motion)

Code:

```
1  #!/usr/bin/python3
2  import OpenGL.GL as GL
3  import math
4
5  from py3d.core.base import Base
6  from py3d.core.utils import Utils
7  from py3d.core.attribute import Attribute
8  from py3d.core.uniform import Uniform
9
10
11 class Example(Base):
12     """ Animate a triangle moving across screen """
13     def initialize(self):
14         print("Initializing program...")
15         # Initialize program #
16         vs_code = """
17             in vec3 position;
18             uniform vec3 translation;
19             void main()
20             {
21                 vec3 pos = position + translation;
22                 gl_Position = vec4(pos.x, pos.y, pos.z, 1.0);
23             }
24         """
25         fs_code = """
26             uniform vec3 baseColor;
27             out vec4 fragColor;
28             void main()
29             {
30                 fragColor = vec4(baseColor.r, baseColor.g, baseColor.b, 1.0);
31             }
32         """
33         self.program_ref = Utils.initialize_program(vs_code, fs_code)
34         # render settings (optional) #
35         # Specify color used when clearing
36         GL.glClearColor(0.0, 0.0, 0.0, 1.0)
37         # Set up vertex array object #
38         vao_ref = GL.glGenVertexArrays(1)
39         GL.glBindVertexArray(vao_ref)
40         # Set up vertex attribute #
41         position_data = [[ 0.0, 0.2, 1],
42                          [ 0.2, -0.2, 0.0],
43                          [-0.2, -0.2, 0.0]]
44         self.vertex_count = len(position_data)
45         position_attribute = Attribute('vec3', position_data)
46         position_attribute.associate_variable(self.program_ref, 'position')
47         # Set up uniforms #
48         self.translation = Uniform('vec3', [-0.5, 0.0, 0.0])
49         self.translation.locate_variable(self.program_ref, 'translation')
50         self.base_color = Uniform('vec3', [1.0, 0.0, 0.0])
51         self.base_color.locate_variable(self.program_ref, 'baseColor')
52
53         self.linear_pos = -1
54         self.orbit_radius = 0.75
55
56     def update(self):
57         """ Update data """
58         t = self.time
59         GL.glClear(GL.GL_COLOR_BUFFER_BIT)
60         GL.glUseProgram(self.program_ref)
61
62         # --- Triangle 1: moves linearly across screen ---
63         self.linear_pos += 0.02
64         if self.linear_pos > 1.2:
65             self.linear_pos = -1.2
66
67         self.translation.data = [self.linear_pos, 0.0, 0.0]
68         self.base_color.data = [1.0, 0.0, 0.0] # Red
69         self.translation.upload_data()
70         self.base_color.upload_data()
71         GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)
72
73         # --- Triangle 2: orbits around origin ---
74         tx = self.orbit_radius * math.cos(t)
75         ty = self.orbit_radius * math.sin(t)
76         self.translation.data = [tx, ty, 0.0]
77         self.base_color.data = [0.0, 0.5, 1.0] # Blue
78         self.translation.upload_data()
79         self.base_color.upload_data()
80         GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)
81
82
83 # Instantiate this class and run the program
84 Example().run()
85
```

Result:

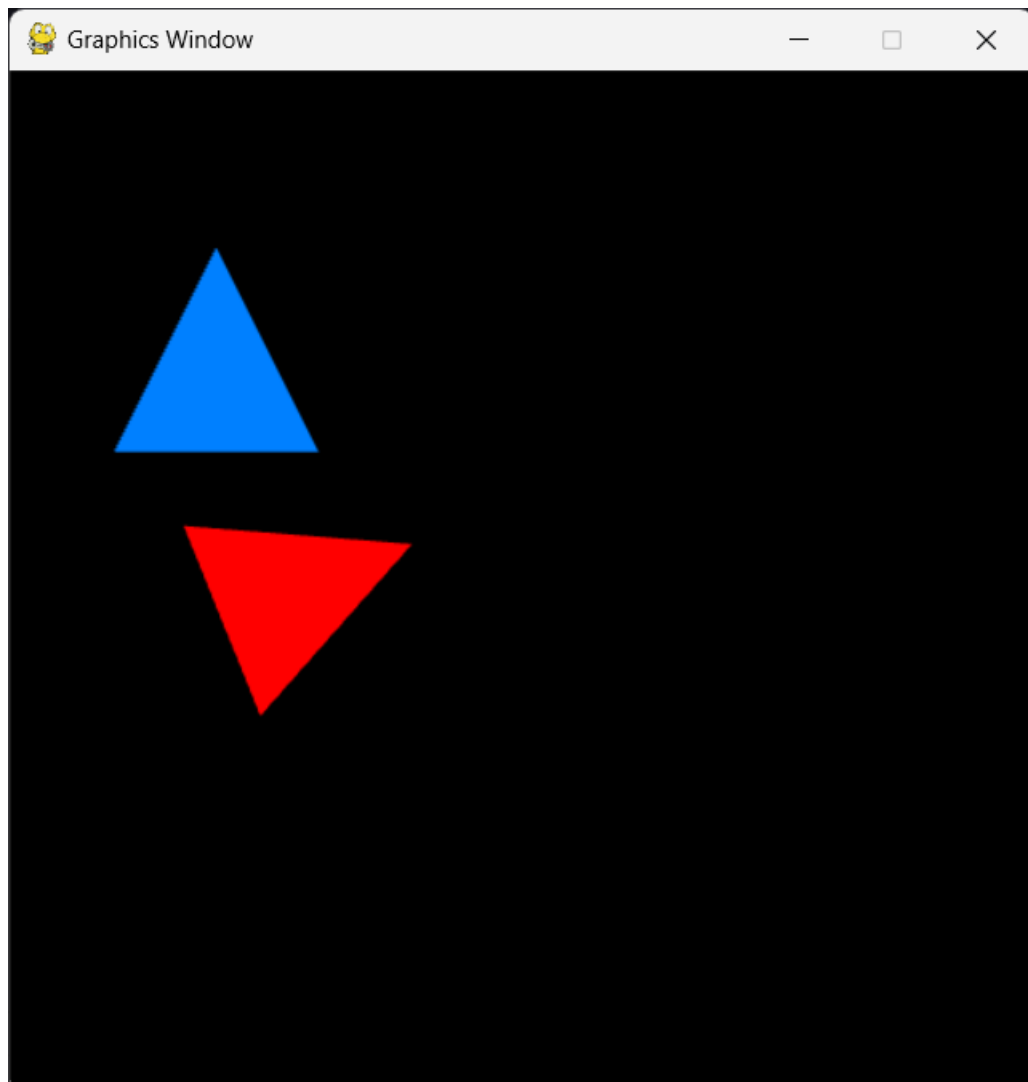


3. Linear motion with Rotation + Circular Motion

Code:

```
1  #!/usr/bin/python3
2  import OpenGL.GL as GL
3  import math
4
5  from py3d.core.base import Base
6  from py3d.core.utils import Utils
7  from py3d.core.attribute import Attribute
8  from py3d.core.uniform import Uniform
9
10
11 class Example(Base):
12     """ Animate a triangle moving across screen """
13     def initialize(self):
14         print("Initializing program...")
15         # Initialize program #
16         vs_code = """
17             in vec3 position;
18             uniform vec3 translation;
19             uniform float angle;
20             void main()
21             {
22                 // rotation matrix
23                 float c = cos(angle);
24                 float s = sin(angle);
25                 mat2 R = mat2(c, -s, s, c);
26
27                 // apply rotation + translation
28                 vec2 rotated = R * position.xy;
29                 vec2 world = rotated + translation.xy;
30
31                 gl_Position = vec4(world, position.z, 1.0);
32             }
33         """
34         fs_code = """
35             uniform vec3 baseColor;
36             out vec4 fragColor;
37             void main()
38             {
39                 fragColor = vec4(baseColor.r, baseColor.g, baseColor.b, 1.0);
40             }
41         """
42         self.program_ref = Utils.initialize_program(vs_code, fs_code)
43         # render settings (optional) #
44         # Specify color used when clearing
45         GL.glClearColor(0.0, 0.0, 0.0, 1.0)
46         # Set up vertex array object #
47         vao_ref = GL.glGenVertexArrays(1)
48         GL.glBindVertexArray(vao_ref)
49         # Set up vertex attribute #
50         position_data = [[ 0.0, 0.2, 1],
51                          [ 0.2, -0.2, 0.0],
52                          [-0.2, -0.2, 0.0]]
53         self.vertex_count = len(position_data)
54         position_attribute = Attribute('vec3', position_data)
55         position_attribute.associate_variable(self.program_ref, 'position')
56         # Set up uniforms #
57         self.translation = Uniform('vec3', [-0.5, 0.0, 0.0])
58         self.translation.locate_variable(self.program_ref, 'translation')
59         self.base_color = Uniform('vec3', [1.0, 0.0, 0.0]) # Red
60         self.base_color.locate_variable(self.program_ref, 'baseColor')
61         self.angle = Uniform('float', 0.0)
62         self.angle.locate_variable(self.program_ref, 'angle')
63
64         self.linear_pos = -1
65         self.orbit_radius = 0.75
66
67     def update(self):
68         """ Update data """
69         t = self.time
70         GL.glClear(GL.GL_COLOR_BUFFER_BIT)
71         GL.glUseProgram(self.program_ref)
72
73         # --- Triangle 1: moves linearly across screen ---
74         self.linear_pos += 0.02
75         if self.linear_pos > 1.2:
76             self.linear_pos = -1.2
77
78         self.translation.data = [self.linear_pos, 0.0, 0.0]
79         self.angle.data = t * 3.0
80         self.base_color.data = [1.0, 0.0, 0.0] # Red
81         self.translation.upload_data()
82         self.angle.upload_data()
83         self.base_color.upload_data()
84         GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)
85
86         # --- Triangle 2: orbits around origin ---
87         tx = self.orbit_radius * math.cos(t)
88         ty = self.orbit_radius * math.sin(t)
89         self.translation.data = [tx, ty, 0.0]
90         self.angle.data = 0.0
91         self.base_color.data = [0.0, 0.5, 1.0] # Blue
92         self.translation.upload_data()
93         self.angle.upload_data()
94         self.base_color.upload_data()
95         GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)
96
97
98 # Instantiate this class and run the program
99 Example().run()
100
```

Result:



Exercise 2: Matrix Algebra and Transformations

Part A – Adding Keyboard Control

```
1  #!/usr/bin/python3
2  from math import pi
3  import OpenGL.GL as GL
4
5  from py3d.core.base import Base
6  from py3d.core_ext.camera import Camera
7  from py3d.core_ext.mesh import Mesh
8  from py3d.core_ext.renderer import Renderer
9  from py3d.core_ext.scene import Scene
10 from py3d.geometry.sphere import SphereGeometry
11 from py3d.material.material import Material
12 from py3d.core.matrix import Matrix
13
14
15 class Example(Base):
16     """ Render a spinning sphere with gradient colors """
17
18     def initialize(self):
19         print("Initializing program...")
20         # --- Setup basic 3D scene ---
21         self.renderer = Renderer()
22         self.scene = Scene()
23         self.camera = Camera(aspect_ratio=800 / 600)
24         self.camera.set_position([0, 0, 7])
25
26         # --- Sphere mesh ---
27         geometry = SphereGeometry(radius=2)
28         vs_code = """
29         uniform mat4 modelMatrix;
30         uniform mat4 viewMatrix;
31         uniform mat4 projectionMatrix;
32         in vec3 vertexPosition;
33         out vec3 position;
34         void main()
35         {
36             vec4 pos = vec4(vertexPosition, 1.0);
37             gl_Position = projectionMatrix * viewMatrix * modelMatrix * pos;
38             position = vertexPosition;
39         }
40         """
41         fs_code = """
42         in vec3 position;
43         out vec4 fragColor;
44         void main()
45         {
46             vec3 color = mod(position, 1.0);
47             fragColor = vec4(color, 1.0);
48         }
49         """
50         material = Material(vs_code, fs_code)
51         material.locate_uniforms()
52         self.mesh = Mesh(geometry, material)
53         self.scene.add(self.mesh)
54
55         # --- Movement parameters ---
56         self.move_speed = 2.0 # units per second
57         self.turn_speed = 90 * (pi / 180) # radians per second
58
59     def update(self):
60         """Update transformations per frame."""
61         move_amount = self.move_speed * self.delta_time
62         turn_amount = self.turn_speed * self.delta_time
63
64         # --- Auto-spin (always active) ---
65         self.mesh.rotate_y(0.00514)
66         self.mesh.rotate_x(0.00377)
67
68         if self.input.is_key_pressed('w'):
69             self.mesh.local_matrix = Matrix.make_translation(0, move_amount, 0) @ self.mesh.local_matrix
70         if self.input.is_key_pressed('s'):
71             self.mesh.local_matrix = Matrix.make_translation(0, -move_amount, 0) @ self.mesh.local_matrix
72         if self.input.is_key_pressed('a'):
73             self.mesh.local_matrix = Matrix.make_translation(-move_amount, 0, 0) @ self.mesh.local_matrix
74         if self.input.is_key_pressed('d'):
75             self.mesh.local_matrix = Matrix.make_translation(move_amount, 0, 0) @ self.mesh.local_matrix
76         if self.input.is_key_pressed('z'):
77             self.mesh.local_matrix = Matrix.make_translation(0, 0, move_amount) @ self.mesh.local_matrix
78         if self.input.is_key_pressed('x'):
79             self.mesh.local_matrix = Matrix.make_translation(0, 0, -move_amount) @ self.mesh.local_matrix
80
81         if self.input.is_key_pressed('i'):
82             self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(0, move_amount, 0)
83         if self.input.is_key_pressed('k'):
84             self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(0, -move_amount, 0)
85         if self.input.is_key_pressed('j'):
86             self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(-move_amount, 0, 0)
87         if self.input.is_key_pressed('l'):
88             self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(move_amount, 0, 0)
89
90         # --- Render frame ---
91         self.renderer.render(self.scene, self.camera)
92
93
94 # Run program
95 Example(screen_size=[800, 600]).run()
96
```

- Explain why global movement uses: $M = T * M$, while local movement uses: $M = M * T$.
- ➔ Global movement uses $M = T * M$ because the transformation T is applied in world coordinates before the object's existing transform, so the object moves relative to the world axes.
- ➔ Local movement uses $M = M * T$ because the transformation T is applied in the object's local coordinates, after its current transform, so the object moves relative to its own axes.

Part B – Interactive Rotations

```

89
90     if self.input.is_key_pressed('q'):
91         self.mesh.local_matrix = Matrix.make_rotation_y(turn_amount) @ self.mesh.local_matrix
92     if self.input.is_key_pressed('e'):
93         self.mesh.local_matrix = Matrix.make_rotation_y(-turn_amount) @ self.mesh.local_matrix
94
95     if self.input.is_key_pressed('u'):
96         self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_rotation_y(turn_amount)
97     if self.input.is_key_pressed('o'):
98         self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_rotation_y(-turn_amount)
99

```

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Explain the difference between rotating around the world axis and rotating around the object's local axis.
- ➔ Rotating around the world axis means the object rotates with respect to the fixed world coordinate system. Its rotation does not depend on the object's current orientation. (Matrix form: $M = R \times M$).
- ➔ Rotating around the object's local axis means the object rotates with respect to its own coordinate system, which moves and turns together with the object. (Matrix form: $M = M \times R$).

Part C – Auto-Spin and Combination

```

1  #!/usr/bin/python3
2  from math import pi
3  import OpenGL.GL as GL
4
5  from py3d.core.base import Base
6  from py3d.core_ext.camera import Camera
7  from py3d.core_ext.mesh import Mesh
8  from py3d.core_ext.renderer import Renderer
9  from py3d.core_ext.scene import Scene
10 from py3d.geometry.sphere import SphereGeometry
11 from py3d.material.material import Material
12 from py3d.core.matrix import Matrix
13
14
15 class Example(Base):
16     """ Render a spinning sphere with gradient colors """
17
18     def initialize(self):
19         print("Initializing program...")
20         # --- Setup basic 3D scene ---
21         self.renderer = Renderer()
22         self.scene = Scene()
23         self.camera = Camera(aspect_ratio=800 / 600)
24         self.camera.set_position([0, 0, 7])
25
26         # --- Sphere mesh ---
27         geometry = SphereGeometry(radius=2)
28         vs_code = """
29         uniform mat4 modelMatrix;
30         uniform mat4 viewMatrix;
31         uniform mat4 projectionMatrix;
32         in vec3 vertexPosition;
33         out vec3 position;
34         void main()
35         {
36             vec4 pos = vec4(vertexPosition, 1.0);
37             gl_Position = projectionMatrix * viewMatrix * modelMatrix * pos;
38             position = vertexPosition;
39         }
40         """
41         fs_code = """
42         in vec3 position;
43         out vec4 fragColor;
44         void main()
45         {
46             vec3 color = mod(position, 1.0);
47             fragColor = vec4(color, 1.0);
48         }
49         """
50         material = Material(vs_code, fs_code)
51         material.locate_uniforms()
52         self.mesh = Mesh(geometry, material)
53         self.scene.add(self.mesh)
54
55         # --- Movement parameters ---
56         self.move_speed = 2.0 # units per second
57         self.turn_speed = 90 * (pi / 180) # radians per second
58
59     def update(self):
60         """Update transformations per frame."""
61         move_amount = self.move_speed * self.delta_time
62         turn_amount = self.turn_speed * self.delta_time
63
64         # --- Auto-spin (always active) ---
65         self.mesh.rotate_y(0.00514)
66         self.mesh.rotate_x(0.00337)
67
68         if self.input.is_key_pressed('w'):
69             self.mesh.local_matrix = Matrix.make_translation(0, move_amount, 0) @ self.mesh.local_matrix
70         if self.input.is_key_pressed('s'):
71             self.mesh.local_matrix = Matrix.make_translation(0, -move_amount, 0) @ self.mesh.local_matrix
72         if self.input.is_key_pressed('a'):
73             self.mesh.local_matrix = Matrix.make_translation(-move_amount, 0, 0) @ self.mesh.local_matrix
74         if self.input.is_key_pressed('d'):
75             self.mesh.local_matrix = Matrix.make_translation(move_amount, 0, 0) @ self.mesh.local_matrix
76         if self.input.is_key_pressed('z'):
77             self.mesh.local_matrix = Matrix.make_translation(0, 0, move_amount) @ self.mesh.local_matrix
78         if self.input.is_key_pressed('x'):
79             self.mesh.local_matrix = Matrix.make_translation(0, 0, -move_amount) @ self.mesh.local_matrix
80
81         if self.input.is_key_pressed('i'):
82             self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(0, move_amount, 0)
83         if self.input.is_key_pressed('k'):
84             self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(0, -move_amount, 0)
85         if self.input.is_key_pressed('j'):
86             self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(-move_amount, 0, 0)
87         if self.input.is_key_pressed('l'):
88             self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(move_amount, 0, 0)
89
90         if self.input.is_key_pressed('q'):
91             self.mesh.local_matrix = Matrix.make_rotation_y(turn_amount) @ self.mesh.local_matrix
92         if self.input.is_key_pressed('e'):
93             self.mesh.local_matrix = Matrix.make_rotation_y(-turn_amount) @ self.mesh.local_matrix
94
95         if self.input.is_key_pressed('u'):
96             self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_rotation_y(turn_amount)
97         if self.input.is_key_pressed('o'):
98             self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_rotation_y(-turn_amount)
99
100        # --- Render frame ---
101        self.renderer.render(self.scene, self.camera)
102
103
104
105 # Run program
106 Example(screen_size=[800, 600]).run()
107

```

Part 2: Homework

Question 1: Spinning Sun

1.

```
self.camera.set_position([0, 0, 7])
geometry = SphereGeometry(radius=1)
```

2.

```
36 void main()
37 {
38     vec3 color = mod(position, 1.0);
39     fragColor = vec4(color, 1.0);
40 }
```

3.

```
46
47 def update(self):
48     self.mesh.rotate_y(0.01)
49     self.renderer.render(self.scene, self.camera)
50
```

Question 2: Adding the Earth (Orbit Only)

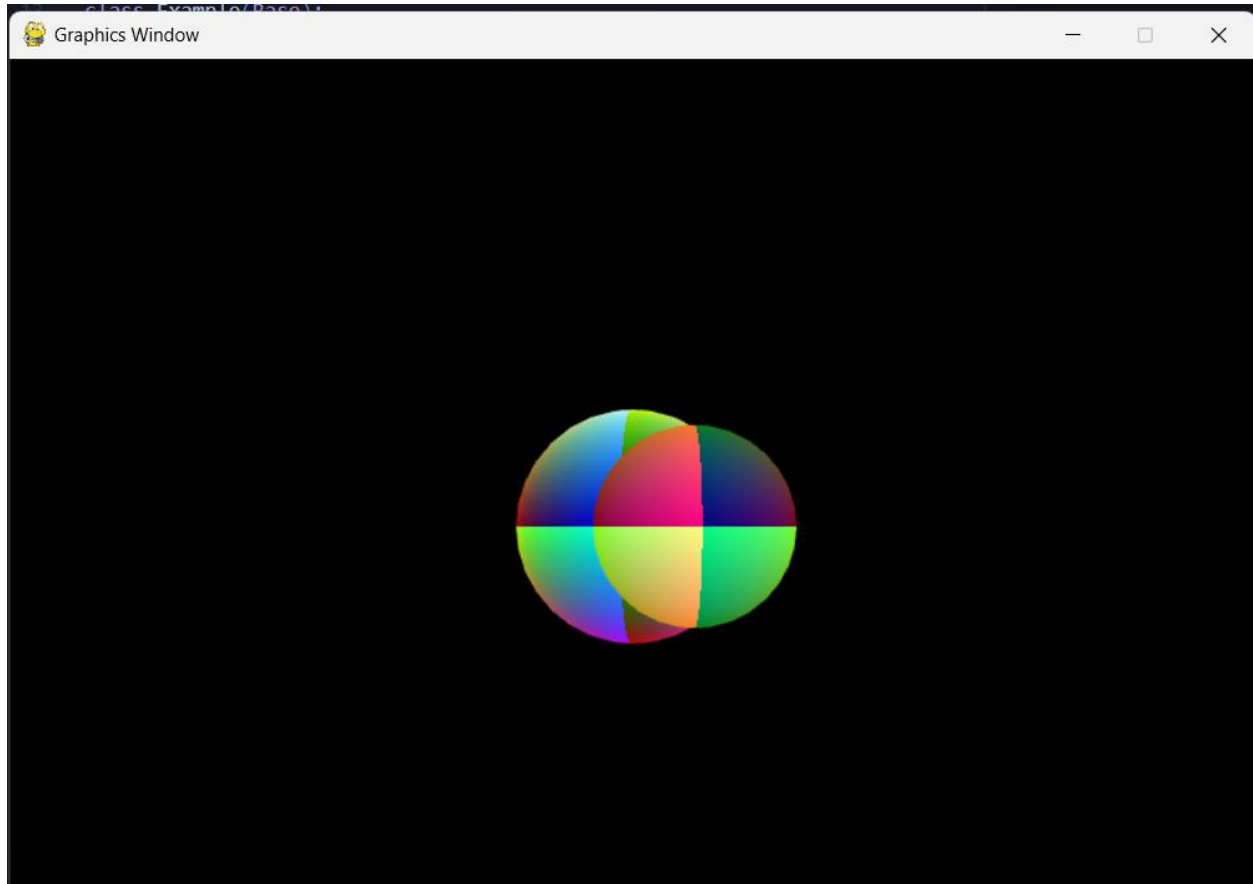
Code:

```

1  #!/usr/bin/python3
2  from math import pi, sin, cos
3  from py3d.core.base import Base
4  from py3d.core_ext.camera import Camera
5  from py3d.core_ext.mesh import Mesh
6  from py3d.core_ext.renderer import Renderer
7  from py3d.core_ext.scene import Scene
8  from py3d.geometry.sphere import SphereGeometry
9  from py3d.material.material import Material
10 from py3d.core.matrix import Matrix
11
12
13 class Example(Base):
14     """ Render a spinning sphere with gradient colors """
15     def initialize(self):
16         print("Initializing program...")
17         self.renderer = Renderer()
18         self.scene = Scene()
19         self.camera = Camera(aspect_ratio=800/600)
20         self.camera.set_position([0, 0, 7])
21         vs_code = """
22         uniform mat4 modelMatrix;
23         uniform mat4 viewMatrix;
24         uniform mat4 projectionMatrix;
25         in vec3 vertexPosition;
26         out vec3 position;
27         void main()
28         {
29             vec4 pos = vec4(vertexPosition, 1.0);
30             gl_Position = projectionMatrix * viewMatrix * modelMatrix * pos;
31             position = vertexPosition;
32         }
33         """
34         fs_code = """
35         in vec3 position;
36         out vec4 fragColor;
37         void main()
38         {
39             vec3 color = mod(position, 1.0);
40             fragColor = vec4(color, 1.0);
41         }
42         """
43         material = Material(vs_code, fs_code)
44         material.locate_uniforms()
45
46         sun_geometry = SphereGeometry(radius=1.0)
47         self.sun = Mesh(sun_geometry, material)
48         self.scene.add(self.sun)
49
50         earth_geometry = SphereGeometry(radius=0.5)
51         self.earth = Mesh(earth_geometry, material)
52         self.scene.add(self.earth)
53
54         self.orbit_radius = 3.0
55         self.orbit_angle = 0.0
56
57     def update(self):
58         self.sun.rotate_y(0.01)
59
60         self.orbit_angle += 0.01
61         x = self.orbit_radius * cos(self.orbit_angle)
62         z = self.orbit_radius * sin(self.orbit_angle)
63         self.earth.set_position([x, 0, z])
64
65         self.renderer.render(self.scene, self.camera)
66
67
68 # Instantiate this class and run the program
69 Example(screen_size=[800, 600]).run()

```

Result:



Question 3: Earth's Self-Spin

1.

```
57     def update(self):
58         self.sun.rotate_y(0.01)
59
60         self.orbit_angle += 0.01
61         x = self.orbit_radius * cos(self.orbit_angle)
62         z = self.orbit_radius * sin(self.orbit_angle)
63         self.earth.set_position([x, 0, z])
64
65         self.earth.rotate_y(0.05)
66         |
67         self.renderer.render(self.scene, self.camera)
68
```


2. The transformation equation for Earth's model matrix is:

$$M_{earth} = T_{orbit} \times R_{self}$$

Where:

$$T_{orbit} = \begin{bmatrix} 1 & 0 & 0 & x \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_{self} = R_y(\theta_{self})$$

3. Explain why both transformations are needed to simulate realistic Earth motion:

- Orbit translation moves the Earth around the Sun — this represents its revolution (the Earth traveling around the Sun once a year).
- Self rotation makes the Earth spin around its own axis — this represents its day/night cycle (the Earth rotating once every 24 hours).

➔ Without self-rotation, Earth would orbit the Sun but always show the same side (like the Moon).

➔ Without orbit translation, Earth would spin in place but not revolve around the Sun.

Question 4: Adding the Moon

1.

```
moon_geometry = SphereGeometry(radius=0.2)
self.moon = Mesh(moon_geometry, material)
self.scene.add(self.moon)

self.orbit_radius = 3.0
self.moon_orbit_radius = 1.0
self.orbit_angle = 0.0
self.moon_orbit_angle = 0.0
```

2.

```
73     self.moon_orbit_angle += 0.05
74     x_moon = x_earth + self.moon_orbit_radius * cos(self.moon_orbit_angle)
75     z_moon = z_earth + self.moon_orbit_radius * sin(self.moon_orbit_angle)
76     self.moon.set_position([x_moon, 0, z_moon])
```

3.

```
78     self.moon.rotate_y(0.1)
```

Result:

